UNIVERSIDAD AUTONOMA DEL ESTADO DE HIDALGO
Instituto de Ciencias Básicas e Ingeniería
Lic. En Ingeniería En Telecomunicaciones
Instalaciones de Telecomunicaciones
REPORTE DE PROYECTO INTEGRADOR
Semestre: 8 Grupo: 1
Julio – Diciembre 2026

SHADOWBOT: vehículo teleoperado de inspección con ESP32, cámara RTSP y visión artificial con YOLO

SHADOWBOT: Teleoperated Inspection Vehicle with ESP32, RTSP Camera, and YOLO-Based Computer Vision

Ángel Gabriel García Rivera *No. Cuenta: 400926*

Abstract:

This paper presents SHADOWBOT, a low-cost remotely operated ground vehicle for inspection and reconnaissance. The system is split into three Wi-Fi nodes: an ESP32-S3 camera board that streams MJPEG video; an ESP32-S3 control board that drives the left and right DC motors of a four mecanum-wheel chassis through BTS7960 H-bridges, a two-axis pan/tilt camera arm and a 16×2 character LCD over I^2C; and an ESP8266 sensor board with an MQ-2 gas/smoke sensor and status LEDs. A landscape Android app built with Flutter runs on a tablet as the operator console, and a DualShock 4 gamepad pairs with the tablet over Bluetooth. A commercial TP-Link Tapo C110 IP camera publishes an RTSP stream that is consumed by a laptop running YOLO detectors; the annotated video is served back to the tablet as MJPEG over HTTP on the local network. The detector recognizes fire, people, animals, vehicles and everyday objects, and a beta automotive-diagnosis mode flags cracks and rust/corrosion. Boards and vision server announce themselves by UDP broadcast, so the app fills in IP addresses automatically. A link watchdog stops the vehicle after 800 ms without commands. On a CPU-only laptop the detector runs at 3–8 inferences per second, while the video reaches the tablet at about 12 fps.

Keywords:

Teleoperated robot, ESP32, IoT, Flutter, RTSP, YOLO, computer vision, object detection, gas sensing

Resumen:

Se presenta SHADOWBOT, un vehículo terrestre de bajo costo, operado a distancia, para tareas de inspección y reconocimiento. El sistema se divide en tres nodos Wi-Fi: una ESP32-S3 con cámara que transmite video MJPEG; una ESP32-S3 de control que mueve los motores de CD de un chasis con cuatro ruedas mecanum mediante puentes H BTS7960, un brazo pan/tilt de dos ejes y una pantalla LCD de caracteres 16×2 por I^2C; y una ESP8266 de sensores con un MQ-2 para gas/humo y LEDs de estado. Una aplicación Android en Flutter, de uso horizontal, corre en una tablet como consola del operador, y un mando DualShock 4 se vincula a la tablet por Bluetooth. Una cámara IP comercial TP-Link Tapo C110 publica un flujo RTSP que recibe una computadora portátil con detectores YOLO; el video ya anotado regresa a la tablet como MJPEG por HTTP dentro de la red local. El detector reconoce fuego, personas, animales, vehículos y objetos de uso común, y un modo beta de diagnóstico automotriz señala fisuras y óxido/corrosión. Las placas y el servidor de visión se anuncian por difusión UDP, por lo que la app llena sola las direcciones IP. Un watchdog de enlace detiene el vehículo si pasan 800 ms sin órdenes. En una laptop sin GPU el detector hace de 3 a 8 análisis por segundo, mientras el video llega a la tablet a unos 12 cuadros por segundo.

Palabras Clave:

Robot teleoperado, ESP32, IoT, Flutter, RTSP, YOLO, visión artificial, detección de objetos, detección de gases

1. Introducción

Muchas tareas de inspección (revisar ductos, bodegas, cuartos de máquinas, zonas con posible fuga de gas o espacios estrechos) exponen a la persona que las realiza. Un vehículo pequeño, operado a distancia y equipado con cámara y sensores, permite observar y medir sin entrar al lugar. Las plataformas comerciales de este tipo suelen ser costosas y cerradas; en cambio, los microcontroladores ESP32 y ESP8266 integran Wi-Fi a muy bajo costo [1], [2], y frameworks como Flutter permiten construir la interfaz de control para Android con poco esfuerzo [3].

En robótica móvil, controlar los motores por lado (izquierdo y derecho) es la configuración más sencilla de construir y de controlar: basta con variar la velocidad de cada lado para avanzar, retroceder y girar sobre su propio eje [4]. SHADOWBOT aplica este esquema sobre un chasis con cuatro ruedas tipo mecanum [5]. El reto en un prototipo de bajo costo no está tanto en la mecánica como en la integración: transmitir video, leer sensores, interpretar la imagen y recibir órdenes al mismo tiempo, con una respuesta suficiente para manejar

con seguridad, y ofrecer al operador una interfaz clara. Repartir esas tareas entre varios dispositivos reduce la carga de cada uno y facilita depurar y reemplazar módulos.

El objetivo de este trabajo fue diseñar e integrar SHADOWBOT, un vehículo de inspección teleoperado que transmite video, mide gas/humo, orienta su cámara con un brazo de dos ejes, reconoce lo que ve mediante visión artificial y se controla desde una aplicación móvil, desde un mando DualShock 4 o desde ambos a la vez. Las aportaciones principales son:

- ➢ Una arquitectura de tres nodos (dos ESP32-S3 y una ESP8266) que se comunican sólo por Wi-Fi y se anuncian solos en la red;
- ➢ Un watchdog de enlace que detiene el vehículo cuando deja de recibir órdenes;
- ➢ La integración de una cámara IP comercial (Tapo C110) mediante RTSP, sin depender de la aplicación del fabricante, y su procesamiento con YOLO en una computadora de la red que regresa a la tablet el video ya anotado;
- ➢ Detección de fuego, personas, animales, vehículos y objetos, y una versión beta de diagnóstico automotriz que señala fisuras y óxido;
- ➢ Una aplicación de consola con video, telemetría, modos de manejo, regreso automático, comandos de voz y análisis de imagen con IA.

1.1 Estado del arte

La teleoperación de vehículos terrestres se ha estudiado desde hace décadas; Fong y Thorpe [6] clasifican sus interfaces y señalan que el operador depende casi por completo del video y la telemetría que recibe. En respuesta a desastres se han usado robots terrestres pequeños para explorar espacios a los que no puede entrar una persona [7], y los robots móviles con sensores de gas para localizar fugas son un área activa de investigación [8]. En la industria, plataformas como ANYmal [9] y Spot [10] hacen rondas de inspección con cámaras y sensores especializados, pero su costo las deja fuera del alcance de escuelas y pequeñas empresas.

Para interpretar la imagen, los detectores de una sola etapa de la familia YOLO [11] localizan y clasifican objetos en una sola pasada de la red, lo que permite trabajar en tiempo casi real incluso sin GPU; la versión YOLO11 de Ultralytics [12] ofrece modelos preentrenados en el conjunto COCO [13] y facilita entrenar modelos propios. SHADOWBOT se ubica en el extremo de bajo costo: no busca autonomía completa, sino ofrecer las funciones básicas de inspección (video, gas, visión artificial y control remoto seguro) con componentes accesibles y una aplicación propia. Algunos ejemplos de robots terrestres teleoperados se muestran en la Figura 1.

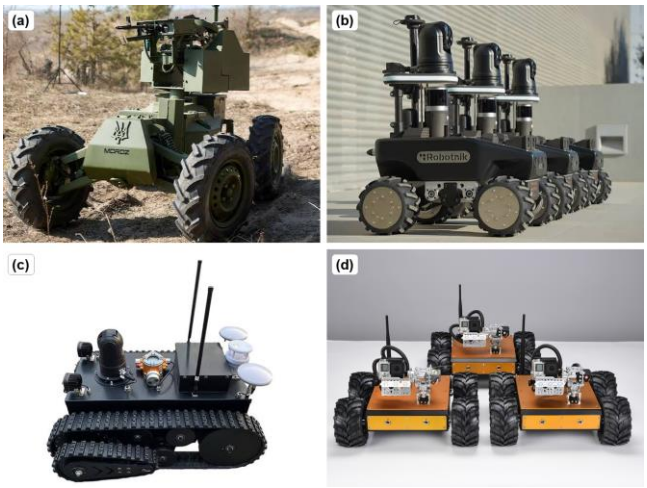


Figura 1. Ejemplos de robots terrestres teleoperados: (a) vehículo terrestre no tripulado de uso militar; (b) robots de inspección y vigilancia de Robotnik con cámara orientable; (c) robot de orugas para inspección con detector de gas; (d) plataformas 4×4 con cámara, luces y brazo pan/tilt. Fuente: imágenes de los fabricantes, recuperadas de Internet.

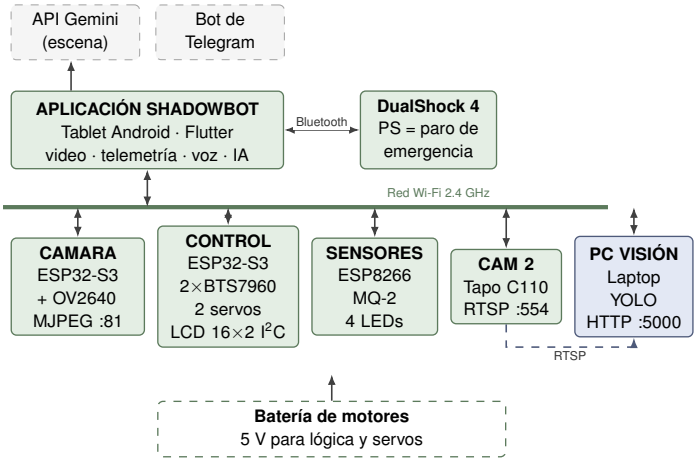


Figura 2. Arquitectura general de SHADOWBOT: nodos CAMARA, CONTROL y SENSORES, cámara Tapo C110, computadora de visión, tablet y mando. Fuente: elaboración propia.

2. Materiales y métodos

2.1 Arquitectura del sistema

El sistema se dividió en tres placas independientes conectadas a la misma red Wi-Fi de 2.4 GHz. Cada una corre su propio firmware (Arduino IDE) y expone servicios HTTP; no comparten pines, sólo la alimentación. El chasis usa cuatro ruedas tipo mecanum y una carcasa que aloja la electrónica y la batería. A la misma red se conectan la cámara IP Tapo C110 [14], que entrega su video por RTSP [15], y una computadora portátil que ejecuta la visión artificial (Sección 2.5). La tablet con la aplicación se comunica con todos por su dirección IP; el mando DualShock 4 se vincula directamente a la tablet por Bluetooth. Los servicios en Internet (Gemini y Telegram) sólo se usan para la descripción de la escena, los avisos de IP y las alertas. La arquitectura general se muestra en la Figura 2 y sus componentes se resumen en la Tabla 1.

Tabla 1. Componentes principales del prototipo. Fuente: elaboración propia.

Nodo	Hardware	Función
CAMARA	ESP32-S3 + OV2640	Video MJPEG (puerto 81), fotos /capture, aviso de IPs por Telegram
CONTROL	ESP32-S3, 2× BTS7960, 2 servos, LCD 16×2 I²C	Tracción por lados, brazo pan/tilt, modo y nivel de gas en el LCD
SENSORES	ESP8266 (NodeMCU), MQ-2, 4 LEDs	Gas/humo y faros
CAM 2	TP-Link Tapo C110	Segunda cámara 1280×720 con visión nocturna, vía RTSP
PC visión	Laptop Ryzen 5 3500U, Python, YOLO	Detección de objetos y video anotado (HTTP, puerto 5000)
App	Tablet Android 8+, Flutter, DualShock 4	Consola del operador, voz e IA

2.2 Firmware de los nodos

- **CONTROL.** Los motores de CD se manejan con dos puentes H BTS7960 [16] usando PWM de 20 kHz y 8 bits (LEDC): un módulo para los motores del lado izquierdo y otro para los del derecho; la velocidad de cada lado se recibe como un valor de -255 a 255. Un watchdog frena el vehículo si pasan 800 ms sin órdenes. Los servos del brazo trabajan a 50 Hz (500–2400 µs) con un perfil de velocidad trapezoidal para moverse sin golpes. El estado se muestra en un LCD de caracteres 16×2 con adaptador I²C PCF8574 [17] (dirección 0x27), conectado sólo con dos hilos de datos: SDA = GPIO14 y SCL = GPIO13. Un controlador propio, en una tarea del núcleo 1, busca la dirección del adaptador y muestra el modo activo (STANDARD, SPORT, EXPLORA o JUEGO) con un ícono animado y la lectura del MQ-2, con el aviso «GAS!» por encima de 35 %. La luz de fondo se enciende o apaga desde la app.
- **CAMARA.** Sirve el flujo MJPEG para la consola y capturas JPEG para fotos y para el análisis con IA. Además recibe por UDP los avisos de los otros nodos y los reenvía a un bot de Telegram (Figura 3).
- **SENSORES.** Es una ESP8266 que lee el MQ-2 [18] por su ADC (con divisor de voltaje), lo reporta en porcentaje y controla los 4 LEDs (faros) en pines contiguos D5–D8.

2.3 Comunicación entre la app y las placas

La app y las placas se comunican por Wi-Fi como si fueran páginas web: la app pide algo (por ejemplo, «avanza») y la placa responde. La orden de movimiento se repite varias veces por segundo; si deja de llegar, el vehículo se detiene solo. La información que se intercambia se muestra en la Tabla 2.

Como el router asigna a cada placa una IP que puede cambiar, CONTROL y SENSORES envían un aviso por difusión UDP (puerto 4210) al conectarse y la app llena sus direcciones automáticamente; la computadora de visión hace lo mismo en el puerto 5001 cada 2 s. Para la Tapo, la app busca en la subred el puerto 554 y confirma usuario y clave con una petición RTSP DESCRIBE. La placa CAMARA reúne también los avisos y un bot de Telegram envía las direcciones (Figura 3); con /ip el bot las repite.

Tabla 2. Información que intercambia cada nodo con la aplicación. Fuente: elaboración propia.

Nodo	La app le envía	El nodo devuelve
CAMARA	Petición de video o de foto	Video en vivo y fotos (también para la IA)
CONTROL	Movimiento, brazo, modo y luz del LCD	Modo activo, señal y tiempo de respuesta
SENSORES	Encender o apagar las luces	Nivel de gas/humo
PC visión	Modo de detección	Video anotado, detecciones y conteo



Figura 3. Bot de Telegram del proyecto enviando las direcciones IP de las placas al conectarse a la red. Fuente: elaboración propia, captura de Telegram.

2.4 Cámara IP Tapo C110 y protocolo RTSP

Además de la cámara de la ESP32-S3, el vehículo lleva una cámara comercial TP-Link Tapo C110 [14], con mayor resolución, visión nocturna y micrófono, montada sobre el brazo pan/tilt. La mayoría de las cámaras IP domésticas están cerradas: su video sólo se ve en la app del fabricante. La Tapo C110 se eligió porque permite activar RTSP [15], un protocolo abierto de video en tiempo real: con una «cuenta de cámara» se obtiene la dirección rtsp://usuario:clave@IP:554/stream2 (1280×720, H.264) que cualquier reproductor compatible puede abrir. Así, la app reproduce su video con media_kit [19] directamente en la red local, sin la nube del fabricante. Las diferencias se resumen en la Tabla 3.

2.5 Flujo de video con visión artificial

Para que el operador vea en la tablet lo que la IA detecta, el video de la Tapo hace un recorrido de tres saltos dentro de la red local (Figura 4):

1) **Cámara → PC (RTSP).** La Tapo codifica el video en H.264 y lo publica por RTSP en el puerto 554. En la laptop, un programa en Python lo abre con OpenCV [20] (FFmpeg) sobre TCP; un hilo lector conserva sólo el cuadro más reciente para que el retraso no se acumule y reabre la conexión si se corta.
2) **Análisis en la PC (YOLO).** Un segundo hilo toma ese cuadro y ejecuta los modelos YOLO del modo activo. Un

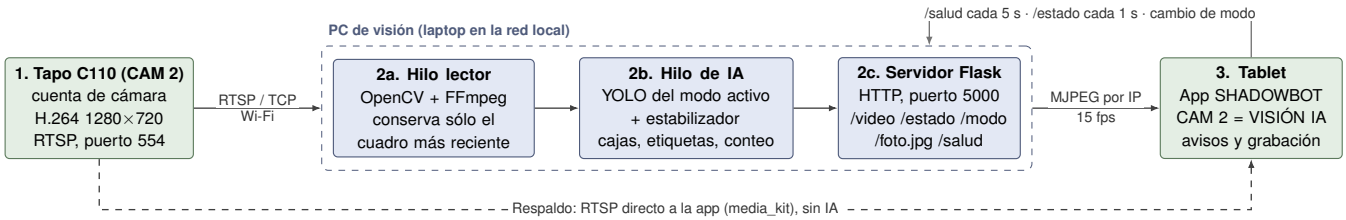


Figura 4. Flujo de video con visión artificial: la Tapo C110 transmite por RTSP a la PC, que analiza cada cuadro con YOLO y envía el video ya anotado a la tablet por HTTP (MJPEG) usando su IP; la ruta punteada es el RTSP directo, que la app usa si la PC no está disponible. Fuente: elaboración propia.

Tabla 3. Diferencia entre una cámara IP cerrada y la Tapo C110 con RTSP. Fuente: elaboración propia.

Característica	Cámara IP cerrada	Tapo C110 con RTSP
Dónde se ve el video	Sólo en la app oficial	Cualquier app compatible
Uso en una app propia	No permitido	Sí, con la dirección RTSP
Conexión	A través de la nube	Directa en la red local
Necesita Internet	Sí	No
Procesar con IA propia	No	Sí (OpenCV/YOLO)

estabilizador reduce falsas alarmas: una detección con confianza menor a 0.60 sólo se acepta si se repite en el análisis siguiente en la misma zona, y una ya confirmada se conserva un ciclo para que el recuadro no parpadee; el fuego nunca se filtra. Sobre cada cuadro se dibujan cajas, etiquetas en español, el conteo por categoría y el modo con su velocidad de análisis.

3) **PC → tablet (HTTP por IP).** Un servidor Flask [21] en el puerto 5000 publica /video (MJPEG, `multipart/x-mixed-replace`) a 15 fps con las últimas detecciones, /estado (JSON con detecciones y conteo), /modo, /foto.jpg y /salud. La app consulta /salud cada 5 s y, en CAM 2, el operador elige entre RTSP directo y VISIÓN IA; si la PC se apaga, la app regresa sola al RTSP y avisa.

Con VISIÓN IA activa, la app lee /estado cada segundo: el fuego muestra un aviso en pantalla con vibración, y las personas o las fisuras y el óxido generan avisos discretos según el modo. El video anotado puede grabarse (archivo con sufijo _IA) y la PC envía alertas con foto por Telegram.

2.6 Modos de detección y diagnóstico automotriz (beta)

La PC tiene cuatro modos que la app cambia en vivo (Tabla 4). El modo SEGURIDAD usa YOLO11s [12] preentrenado en COCO [13] a 480 px, con conteo de personas, animales (perro, gato, ave, caballo, vaca, oveja), vehículos (auto, moto, autobús, camión, bicicleta) y objetos de interés (mochila, bolso, maleta, celular, laptop, cuchillo); el fuego se detecta con un modelo YOLO especializado.

El modo INSPECCIÓN es la versión beta de diagnóstico automotriz: con el brazo pan/tilt, la cámara recorre chasis, carrocería o estructuras y dos modelos adicionales señalan fisuras (segmentación) y óxido/corrosión en tres niveles (leve, moderado, severo). Para evitar confusiones, una zona de «óxido» que se encima con una llama se descarta. Se considera beta porque los modelos se entrenaron con pavimento y estructuras, no con piezas automotrices, y aún no se valida en vehículos reales.

Tabla 4. Modos de detección y velocidad de análisis medida en la laptop sin GPU. Fuente: elaboración propia.

Modo	Qué detecta (modelos)	Análisis/s
FUEGO	Fuego (modelo dedicado)	≈8
SEGURIDAD	Fuego + personas, animales, vehículos y objetos (YOLO11s, COCO)	≈3–5
INSPECCIÓN (beta automotriz)	Fuego + fisuras + óxido/corrosión	≈3
TODO	Todos los modelos	≈2

2.7 Modos de operación y seguridad

Se definieron cuatro modos de manejo: MENÚ (sólo se elige el modo), SPORT (velocidad máxima, manejo tipo carrito RC), EXPLORACIÓN (velocidad ajustable de 20 a 100 %, tablero de gas, escaneo con la cámara y modo preciso al 35 %) y DANCE (coreografías y bailes con música en la tablet). Dentro de EXPLORACIÓN, el submodo AUTÓNOMO graba los tramos que manda el operador y puede regresar solo por la misma ruta en orden inverso. El modo es el mismo en la app, el mando y el LCD.

La seguridad se resolvió en dos niveles. Primero, el mando y la app quedan bloqueados hasta que el operador presiona INICIAR. Segundo, el botón PS del mando es un paro de emergencia en cualquier modo, y el watchdog de 800 ms frena el vehículo si se pierde el enlace. Las funciones secundarias se agrupan en combinaciones con L2 mantenido (Tabla 5).

2.8 Aplicación móvil

La aplicación se desarrolló en Flutter [3] con estética de consola militar en orientación horizontal: cámaras al centro, brazo y modos a la izquierda, y telemetría y joystick a la derecha, más una vista de pantalla completa con filtros de imagen y una sección Multimedia con fotos y videos. En la sección Sistema se configuran la visión IA, la calibración de motores y la clave de la API de Gemini. Para la IA generativa, la app envía una captura a Gemini Flash [22], que devuelve una descripción de la escena que se muestra y se lee en voz

Tabla 5. Combinaciones del mando DualShock 4 (L2 mantenido). Fuente: elaboración propia.

Combinación	Acción
L2 + R1	Cambiar cámara principal (CAM 1 ↔ CAM 2)
L2 + ✕	Encender/apagar faros (4 LEDs)
L2 + ◯	Retroceso corto
L2 + □	Siguiente filtro: normal, IR, B/N, verde
L2 + △	Analizar la imagen con IA (Gemini)
L2 + Share	Grabar/detener video
L2 + Options	Luz del LCD
L2 + L3 / R3	Bajar/subir velocidad (10 %)
L2 + ↑/→/↓ / ←	Brazo a la posición A/B/C / centrar
PS	Paro de emergencia

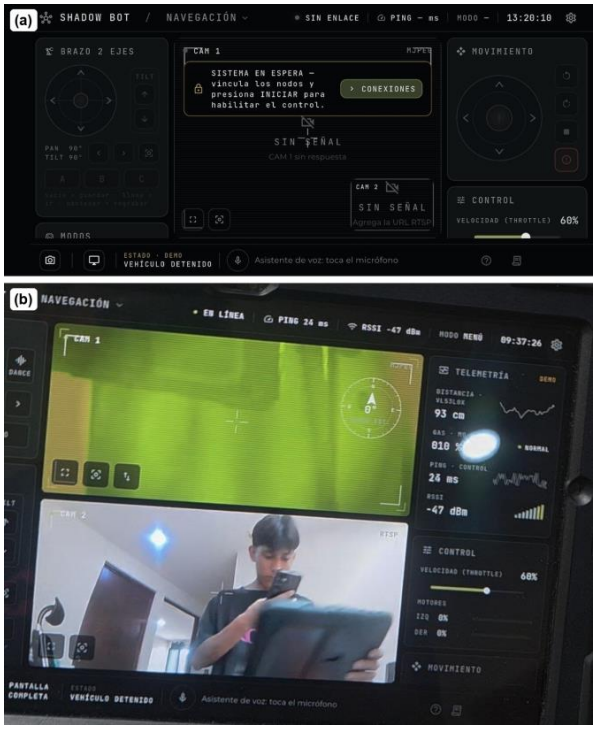


Figura 5. Aplicación SHADOWBOT: (a) consola en espera, antes de vincular los nodos (captura de pantalla); (b) consola funcionando en una tablet con las dos cámaras y la telemetría. Fuente: elaboración propia.

alta; también hay comandos de voz para mover el vehículo, cambiar de modo o pedir el análisis. La interfaz se muestra en la Figura 5.

3. Resultados y discusión

Se integró el prototipo (Figura 6). Las placas se anuncian por UDP y la app llena solas sus direcciones; también llegan a Telegram cada vez que se enciende el vehículo. La app muestra el estado de cada nodo, el video con filtros, la lectura del MQ-2 y el control de los LEDs, y el LCD de CONTROL refleja el modo y el gas. Los tres firmwares caben en la partición por defecto de la memoria flash.

La Tapo C110 se mostró en la tablet tanto por RTSP directo como a través de la PC de visión, sin la app ni la nube del fabricante. En una laptop con Ryzen 5 3500U sin GPU, YOLO hace de 2 a 8 análisis por segundo según el modo

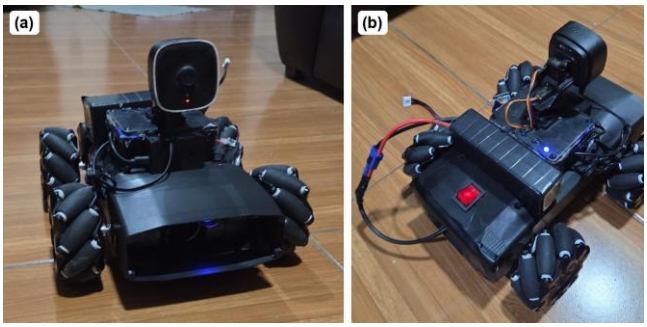


Figura 6. Prototipo de SHADOWBOT: (a) vista frontal; (b) vista superior con el brazo de dos servos, el interruptor de encendido y la conexión de la batería. Fuente: elaboración propia.

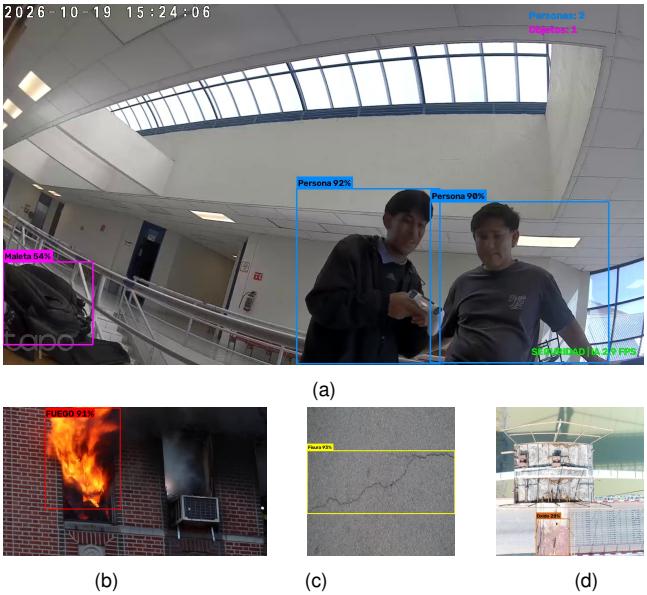


Figura 7. Detecciones de la PC de visión: (a) modo SEGURIDAD con la Tapo C110 en la escuela (personas y objeto); (b) fuego; (c) fisura y (d) óxido del modo de diagnóstico automotriz beta, sobre imágenes de prueba. Fuente: elaboración propia.

(Tabla 4), pero como la captura, el análisis y el envío corren en hilos separados, el video anotado llega a la tablet a unos 12 fps sin atrasarse. En pruebas en la escuela, el modo SEGURIDAD reconoció personas con 90–92 % de confianza y una maleta con 54 % (Figura 7a). Se observó que la cámara debe ir derecha: con la Tapo de lado, YOLO no reconocía personas, y al enderezarla la confianza subió a 88 %. Con imágenes de prueba, el modelo de fuego alcanzó 91 % y el de fisuras 93 %; el de óxido sólo 28 %, lo que confirma que el diagnóstico automotriz debe seguir como beta.

Separar el sistema en nodos hizo cada firmware más pequeño y fácil de depurar, y llevar la IA a una computadora externa evitó cargar a los microcontroladores. La desventaja es que todo depende de la red Wi-Fi y de que la PC esté encendida; por eso la app conserva siempre la vista RTSP directa como respaldo.

Como trabajo futuro se propone medir la latencia de extremo a extremo y la autonomía de la batería, calibrar el MQ-2 [18], entrenar un modelo propio con imágenes de chasis, carrocería y llantas, complementar el diagnóstico automotriz

con la lectura de códigos de falla por OBD-II, ejecutar YOLO en un equipo con acelerador y agregar un punto de acceso propio en el vehículo.

4. Conclusiones

SHADOWBOT muestra que es posible construir un vehículo de inspección completo con dos ESP32-S3, una ESP8266 y una aplicación móvil: video en vivo, brazo de cámara, sensado de gas, control redundante (app y mando) con paro de emergencia y visión artificial. El flujo Tapo $\rightarrow$ RTSP $\rightarrow$ PC con YOLO $\rightarrow$ HTTP $\rightarrow$ tablet permitió añadir detección de fuego, personas, animales, vehículos y objetos sin modificar el firmware de las placas, y sienta la base de un diagnóstico automotriz que, en su versión beta, ya señala fisuras y óxido. Además, las direcciones IP se descubren solas en la red. Elegir cámaras con protocolos abiertos permite aprovechar hardware comercial dentro de una aplicación propia y procesarlo con IA local.

Declaración sobre el uso de inteligencia artificial

El autor declara que se utilizaron herramientas de IA generativa como apoyo en la redacción del manuscrito, en la elaboración de los diagramas y en el desarrollo del software. El sistema usa la API de Gemini para describir imágenes y modelos YOLO para la detección de objetos.

Referencias

[1] Espressif Systems, "ESP32-S3 Series Datasheet," 2023. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[2] Espressif Systems, "ESP8266EX Datasheet," 2023. [Online]. Available: https://www.espressif.com

[3] Google, "Flutter documentation." [Online]. Available: https://docs.flutter.dev

[4] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

[5] B. E. Ilon, "Wheels for a course stable selfpropelling vehicle movable in any desired direction on the ground or some other base," U.S. Patent 3 876 255, Apr. 8, 1975.

[6] T. Fong and C. Thorpe, "Vehicle teleoperation interfaces," *Autonomous Robots*, vol. 11, no. 1, pp. 9–18, 2001.

[7] R. R. Murphy, *Disaster Robotics*. Cambridge, MA, USA: MIT Press, 2014.

[8] A. J. Lilienthal, A. Loutfi, and T. Duckett, "Airborne chemical sensing with mobile robots," *Sensors*, vol. 6, no. 11, pp. 1616–1678, 2006.

[9] M. Hutter et al., "ANYmal - a highly mobile and dynamic quadrupedal robot," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2016, pp. 38–44.

[10] Boston Dynamics, "Spot." [Online]. Available: https://bostondynamics.com/products/spot/

[11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[12] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[13] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[14] TP-Link, "Tapo C110 Home Security Wi-Fi Camera," product specifications. [Online]. Available: https://www.tapo.com

[15] H. Schulzrinne, A. Rao, and R. Lanphier, "Real Time Streaming Protocol (RTSP)," IETF RFC 2326, Apr. 1998.

[16] Infineon Technologies, "BTN7960 High Current PN Half Bridge NovalithIC," Data Sheet. [Online]. Available: https://www.infineon.com

[17] NXP Semiconductors, "PCF8574; PCF8574A Remote 8-bit I/O expander for I^2C-bus with interrupt," Data Sheet, rev. 5, 2013.

[18] Hanwei Electronics, "Technical Data MQ-2 Gas Sensor," Data Sheet. [Online]. Available: https://www.pololu.com/file/0J309/MQ2.pdf

[19] media-kit, "media_kit: cross-platform video and audio player for Flutter and Dart," GitHub repository. [Online]. Available: https://github.com/media-kit/media-kit

[20] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[21] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com

[22] Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev/gemini-api/docs